



Security Audit

UNIT Technologies 2nd (RWA)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Code Quality	9
Audit Resources	9
Dependencies	9
Severity Definitions	10
Status Definitions	11
Audit Findings	12
Centralisation	51
Conclusion	52
Our Methodology	53
Disclaimers	55
About Hashlock	56

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The UNIT Technologies team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

UNIT is a decentralized finance (DeFi) protocol designed to make stablecoin liquidity productive through a tokenized deposit and staking model. The protocol allows users to deposit supported stablecoins and receive UNIT, a stable asset backed 1:1 by the underlying stablecoins deposited into the protocol. UNIT serves as the primary liquid asset of the ecosystem and can be held, transferred, or used within supported DeFi applications.

The protocol separates the liquid representation of deposited assets from participation in yield generation through sUNIT (Staked UNIT). Users can stake UNIT to receive sUNIT, which represents their staked position in the protocol and provides exposure to rewards generated from the underlying stablecoin capital. This structure allows UNIT to remain liquid while users who choose to stake can participate in the yield generated by the protocol.

Project Name: UNIT Technologies

Project Type: RWA

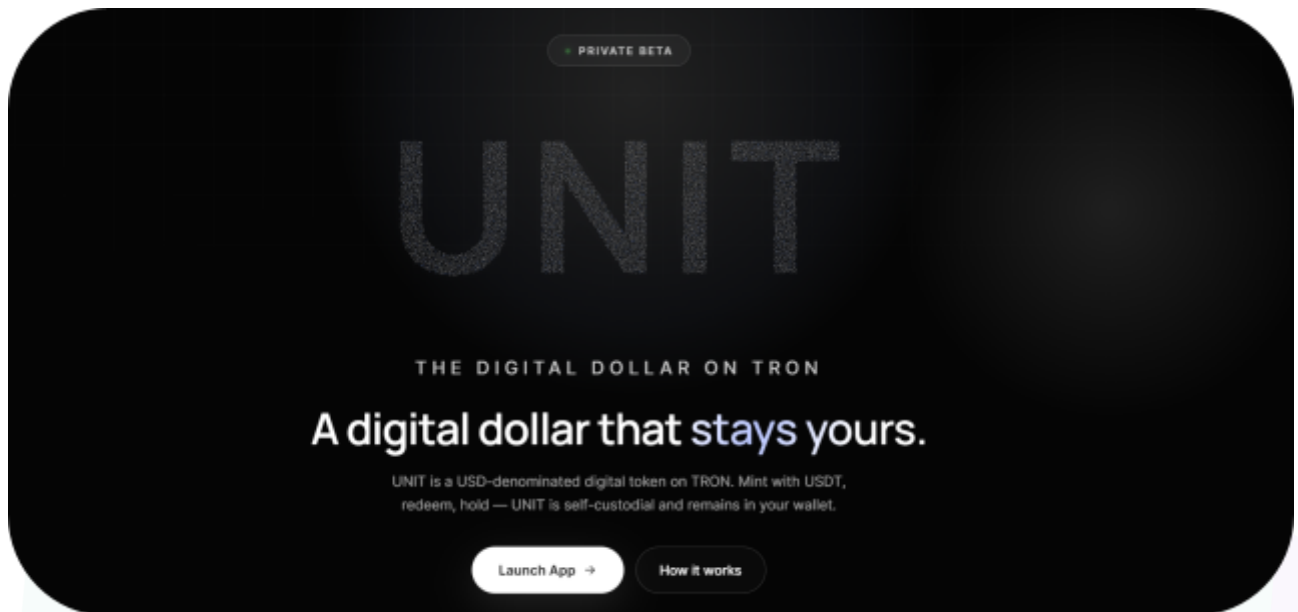
Compiler Version: ^0.8.26

Website: unitdefi.com

Logo:



Project Visuals:



1:1

USDT redemption for every UNIT

Self-custodial

UNIT stays in your wallet

Screened

Every deposit, before mint

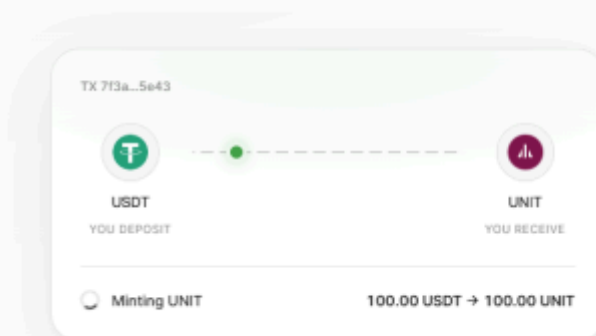
TRC-20

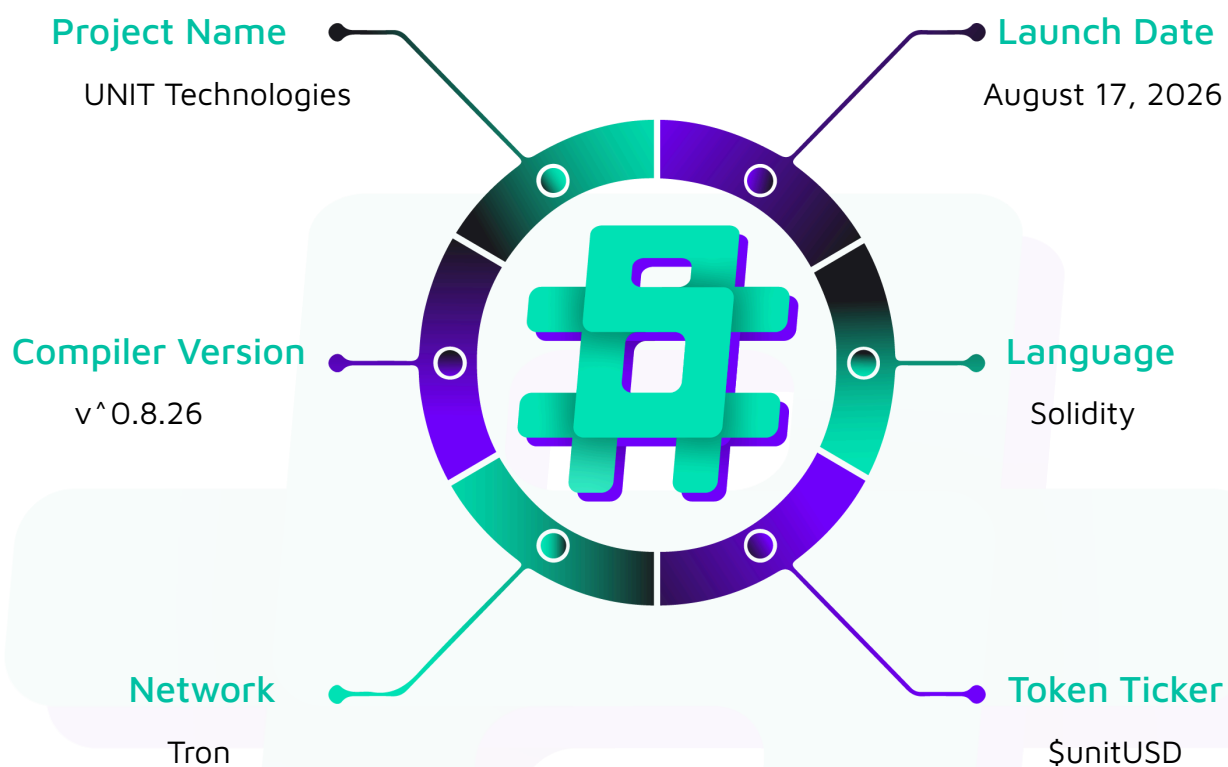
Native on the TRON blockchain

— INSTANT

A mint settles in seconds

Deposit USDT, clear screening, and UNIT lands in your wallet — fully on-chain, with no waiting on intermediaries.



Visualised Context:

Audit Scope

We at Hashlock audited the solidity code within the UNIT Technologies project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	UNIT Technologies Smart Contracts
Network	TRON
Language	Solidity
Audit Date	September, 2026
Contract 1	Minter2.sol
Contract 2	StakedUnit.sol
Contract 3	Unit.sol
Contract 4	CumulativeMerkleDrop.sol
Audited GitHub Commit Hash	09627be72029f2372e301e63c9dc89f24b274c22

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

2 Medium severity vulnerabilities

3 Low severity vulnerabilities

2 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Code Quality

This audit scope involves the smart contracts of the UNIT Technologies project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the UNIT Technologies project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] Minter2#_depositToJustLend - Total-backing invariant rejects Compound share rounding and wedges a fresh deployment

Description

JustLend's jUSDD is a Compound v2 CErc20Delegator with 8-decimal shares and an exchange-rate mantissa of roughly $1.0013e26$. Its `mint()` floors the shares it issues (`mintTokens = floor(amount * 1e18 / rate)`), so the value credited to the depositor is short of the deposit by up to `ceil(rate / 1e18)` wei, about $1.0013e8$ wei of USDD at today's rate. The per-deposit rounding check in `_depositToJustLend()` tolerates exactly this loss, but the total-backing check that follows it at `src/Minter2.sol:219-220` does not: it requires the freshly credited value plus everything already held to cover the full nominal deposit plus all outstanding UNIT. With zero surplus there is nothing to absorb the rounding, and both `mint()` and `claimAndDistributeRewards()` revert with `OperationFailed`.

Vulnerability Details

`_depositToJustLend()` is the single sink for every USDD that enters Minter2. It is reached from `_mintInternal()` (the `mint()` path) and from `claimAndDistributeRewards()`. The relevant code at `src/Minter2.sol:209-222`:

```
uint256 rate = jUSDD.exchangeRateStored();
uint256 creditedBacking = Math.mulDiv(sharesReceived, rate, 1e18);
uint256 nominalUnits = usddAmount / 1e12;
uint256 nominalBacking = nominalUnits * 1e12;

if (nominalBacking > creditedBacking) {
    uint256 loss = nominalBacking - creditedBacking;
    if (loss > Math.min(Math.ceilDiv(rate, 1e18), 1e12 - 1)) revert
    OperationFailed(); // tolerates rounding
```

```

}

uint256 backing = USDD.balanceOf(address(this)) +
Math.mulDiv(jUSDD.balanceOf(address(this)), rate, 1e18);
if (backing < (UNIT.totalSupply() + nominalUnits) * 1e12) revert OperationFailed();
// does not

return nominalUnits;

```

The first check (lines 214-217) explicitly accepts a per-deposit loss of up to `ceil(rate / 1e18)` wei. The second check (lines 219-220) then demands that `backing >= (UNIT.totalSupply() + nominalUnits) * 1e12`. On a contract with no pre-existing surplus, `backing` equals `creditedBacking`, which by construction of the first branch can be below `nominalBacking` by the tolerated amount. The invariant therefore fails on precisely the inputs the previous line was written to allow. The two checks are mutually inconsistent whenever the surplus held by the contract is smaller than one deposit's rounding loss.

The existing test suite is green because `test/MockjUSDD.sol` starts at an exchange rate of exactly `1e18` and every test amount divides it exactly, so `creditedBacking == nominalBacking` in every existing test. The mainnet rate is a large non-round integer (`100126903694819958818088090` on 2026-09-22) that changes on every interest accrual, so the rounding loss is essentially always non-zero in production.

All of the following were reproduced in `test/AuditPoCs.t.sol` against a Compound-faithful `jUSDD` mock initialised with the live exchange rate:

- A 100 USDT mint on a fresh deployment loses 44,052,930 wei to share rounding. The per-deposit check passes; the total-backing check reverts `mint()` with `OperationFailed`.
- Fuzzing mint amounts from 1 to `1e9` USDT on a fresh deployment (256 runs) reverts exactly when Compound rounding loses at least 1 wei, and succeeds otherwise.
- With a surplus seeded to three times the per-deposit tolerance and zero yield, ten consecutive mints of 1,000 to 1,063 USDT succeed 6 times and then stall: each mint consumes a slice of surplus that nothing replenishes.

- Any UNIT supply this instance does not back (for example the 25,122 UNIT minted by the previous Minter, or an admin recapitalising sUNIT through a direct Unit.mint) makes both mint() and claimAndDistributeRewards() revert, because the invariant reads the global UNIT.totalSupply().

Live status: the deployed Minter2 already runs this code (its bytecode contains the 1e12 - 1 tolerance constant introduced in commit 596cc68) and currently holds about 25,335 USDD of jUSDD against 25,122.5 UNIT, a surplus of about 212.6 USDD. It is not affected today because that surplus absorbs the rounding. The defect is a regression introduced by commit 596cc68, which added the total-backing check on top of the per-deposit tolerance without reconciling the two.

Proof of Concept

The following is adapted from test_M1_firstMintRevertsOnFreshDeployment, test_M1_dustDonationUnblocksFirstMint and test_M1_unbackedSupplyBlocksMintAndRewardClaims in test/AuditPoCs.t.sol. jUSDD is a Compound-faithful mock at the live rate 0x52d2b2535486fc24b0249a, the PSM has tin = tout = 0, and Minter2 is freshly deployed with no surplus.

```
// jUSDD.exchangeRateStored() on TRON mainnet, 2026-09-22
uint256 constant MAINNET_RATE = 0x52d2b2535486fc24b0249a;

function _expectedLoss(uint256 usddAmount) internal pure returns (uint256) {
    uint256 shares = (usddAmount * 1e18) / MAINNET_RATE;
    return usddAmount - (shares * MAINNET_RATE) / 1e18;
}

function test_M1_firstMintRevertsOnFreshDeployment() public {
    // 100 USDD -> 44,052,930 wei lost to Compound share flooring; per-deposit
    // tolerance is 100,126,904 wei
    usdt.mint(userA, 100e6);
    vm.startPrank(userA);
    usdt.approve(address(minter2), 100e6);
    bytes memory sig = _sig(minter2.MINT_TYPEHASH(), userA, 100e6, false,
    block.timestamp + 1 hours);
    vm.expectRevert(Minter2.OperationFailed.selector);
```

```

    minter2.mint(100e6, false, 0, block.timestamp + 1 hours, sig); // total-backing
check fails
    vm.stopPrank();
    assertEq(UNIT.totalSupply(), 0);
}

function test_M1_dustDonationUnblocksFirstMint() public {
    uint256 loss = _expectedLoss(100e18);
    assertGt(loss, 0);
    assertLe(loss, (MAINNET_RATE + 1e18 - 1) / 1e18); // inside the per-deposit
tolerance: it is the total check that fails
    usdd.mint(address(minter2), loss); // seed exactly the rounding
loss
    _mintFor(userA, 100e6, false); // now succeeds
    assertEq(UNIT.balanceOf(userA), 100e6);
}

function test_M1_unbackedSupplyBlocksMintAndRewardClaims() public {
    vm.prank(admin);
    UNIT.mint(userB, 25_122e6); // supply this instance does not back (previous
Minter, or sUNIT recapitalisation)

    usdt.mint(userA, 100e6);
    vm.startPrank(userA);
    usdt.approve(address(minter2), 100e6);
    bytes memory sig = _sig(minter2.MINT_TYPEHASH(), userA, 100e6, false,
block.timestamp + 1 hours);
    vm.expectRevert(Minter2.OperationFailed.selector);
    minter2.mint(100e6, false, 0, block.timestamp + 1 hours, sig);
    vm.stopPrank();

    // The reward pipeline is blocked by the same invariant.
    vm.etch(minter2.JUSTLEND_DISTRIBUTOR(), address(new
MockClaimDistributor(IERC20(USDD_ADDR))).code);
    usdd.mint(minter2.JUSTLEND_DISTRIBUTOR(), 1_000e18);
    uint256[] memory amounts = new uint256[](1);
    amounts[0] = 50e18;
    IMultiMerkleDistributor.ClaimParam[] memory claims = new

```

```
IMultiMerkleDistributor.ClaimParam[](1);
    claims[0] = IMultiMerkleDistributor.ClaimParam({merkleIndex: 0, index: 0,
amounts: amounts, merkleProof: new bytes32[] (0)});
    vm.prank(admin);
    vm.expectRevert(Minter2.OperationFailed.selector);
    minter2.claimAndDistributeRewards(claims, distributor);
}
```

Run the full M-1 reproduction set with logged loss values:

```
forge test --match-path test/AuditPoCs.t.sol --match-test M1 -vv
```

The five tests are `test_M1_firstMintRevertsOnFreshDeployment`, `test_M1_dustDonationUnblocksFirstMint`, `testFuzz_M1_firstMintRevertsIffRoundingLoss`, `test_M1_surplusDrainsWithoutYield` and `test_M1_unbackedSupplyBlocksMintAndRewardClaims`. All pass, confirming the revert conditions described above.

Impact

The primary user function and the reward pipeline fail deterministically on every fresh deployment or migration until someone donates USDD or jUSDD to the contract. In the launch state of any new `Minter2`, no user can mint and no keeper can run `claimAndDistributeRewards()`.

After a `withdraw()` yield sweep that takes the entire surplus, mints fail until interest re-accrues. At the current jUSDD supply rate of about 0.00085 percent APY, one block of accrual on the live TVL exceeds one mint's rounding loss, so the condition self-heals at today's scale. It does not self-heal at small TVL, where a block of interest is below one deposit's loss.

Any UNIT minted by another `MINTER_ROLE` holder re-triggers the condition, because the invariant is checked against the global `UNIT.totalSupply()` rather than the supply this instance backs. Routine admin actions such as recapitalising the staking vault or running two minters in parallel therefore halt both `mint()` and the reward pipeline.

No funds are at risk and no value can be extracted. Recovery is a dust-level transfer of USDD or jUSDD to the contract.

Severity is Medium because core functionality is broken in the default state of every new deployment, the reward pipeline can be halted by ordinary admin actions, and the test suite actively hides the defect behind a round-number mock. It is not High because there is no fund-loss path and the fix is a trivial donation. It is not Low because this is not an edge case; it is the launch state of the contract.

Recommendation

Let the invariant absorb a bounded, dust-level deficit so that a zero-surplus state cannot wedge the contract. At roughly $1e8$ wei lost per mint, reaching a $1e12$ wei ceiling (0.000001 USD) would take about 10,000 mints with zero yield, and any interest accrual resets the drift.

```
-      uint256 backing = USDD.balanceOf(address(this)) +
Math.mulDiv(jUSDD.balanceOf(address(this)), rate, 1e18);
-      if (backing < (UNIT.totalSupply() + nominalUnits) * 1e12) revert
OperationFailed();
+      uint256 backing = USDD.balanceOf(address(this)) +
Math.mulDiv(jUSDD.balanceOf(address(this)), rate, 1e18);
+      uint256 required = (UNIT.totalSupply() + nominalUnits) * 1e12;
+      // absorb bounded Compound share rounding (< 1e-6 USD) instead of wedging
the contract
+      if (backing + 1e12 < required) revert OperationFailed();
```

If the strict check is preferred for accounting reasons, keep it but make the seeding requirement explicit and enforced: replace `test/MockjUSDD.sol` with a Compound-faithful mock (8-decimal shares, non-round exchange rate such as the live `100126903694819958818088090`) and add a fuzz test over mint amounts so the first-mint revert is visible in CI rather than discovered on mainnet.

Independently of the code change, add the following to the deployment and operations runbook:

- Seed every new deployment with a small USDD buffer (a few $1e12$ wei is ample)



before granting `SIGNER_ROLE`, so the first mint can clear the invariant.

- Never sweep the full surplus with `withdraw()`. Leave at least one deposit's rounding loss behind.
- When migrating, move all `jUSDD` from the old instance into the new one before enabling mints, so the new instance backs the supply it inherits.
- Add backing before any `UNIT` mint that does not pass through this `Minter2`, since the invariant reads global supply.
- Decide explicitly whether the invariant should read the global `UNIT.totalSupply()` at all. Coupling this instance to supply it does not control is what turns routine admin actions elsewhere into a `Minter2` outage. A per-instance `mintedByThis` counter would keep the solvency guarantee without that coupling.

Status

Resolved

[M-02] StakedUnit#redeem - A staker can front-run a pending freeze or confiscation and move the full position to a fresh address

Description

`StakedUnit._withdraw` checks the current frozen state of `caller`, `receiver` and `owner`, and `redeem` and `withdraw` pay the underlying UNIT out immediately to any `receiver`. `Unit.setFrozen`, `Unit.confiscate` and `StakedUnit.confiscate` are ordinary public transactions with no preceding lock, delay or pending state. A holder who observes the administrator's freeze or confiscation transaction before it is included can redeem the entire sUNIT position to a fresh, unfrozen address ahead of it, so the administrative action lands on an empty account. The post-freeze guards themselves hold, as documented in the QA finding on `Unit._update`; the gap is the window before the freeze takes effect.

Vulnerability Details

The vault's exit hook evaluates the freeze flags at the moment of the call and nowhere else, at `src/StakedUnit.sol:67-75`:

```
function _withdraw(address caller, address receiver, address owner, uint256 assets,
uint256 shares)
    internal
    override
{
    if (!_isSolvent()) revert VaultInsolvent();
    Unit unit = Unit(address(asset()));
    if (unit.isFrozen(caller) || unit.isFrozen(receiver) || unit.isFrozen(owner))
revert AccountFrozen();
    super._withdraw(caller, receiver, owner, assets, shares);
}
```

The freeze itself is a single public state write with immediate effect and no pending phase, at `src/Unit.sol:28-36`:

```
function setFrozen(address account, bool frozen) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    if (account == address(0)) revert ZeroAddress();
```

```

    isFrozen[account] = frozen;
    if (frozen) {
        emit Frozen(account);
    } else {
        emit Unfrozen(account);
    }
}

```

The race, step by step:

- The administrator broadcasts `UNIT.setFrozen(holder, true)`, or `sUNIT.confiscate(holder, treasury, shares)`, or both.
- The holder, or a bot acting for the holder, broadcasts `sUNIT.redeem(allShares, fresh, holder)` and has it ordered first. At that moment no address involved is frozen, so the `_withdraw` check passes. `super._withdraw` burns the shares through `StakedUnit._update(holder, address(0), shares)`, which the non-transferability rule at `src/StakedUnit.sol:77-82` allows because `to` is zero, and then pays UNIT through `IERC20(asset()).safeTransfer(fresh, assets)`, which reaches `Unit._update(sUNIT, fresh, assets)` with two unfrozen parties.
- The administrator's transaction is then executed. `setFrozen` flags an account holding zero UNIT and zero sUNIT. `StakedUnit.confiscate(holder, ...)` reverts inside `_burn` with `ERC20InsufficientBalance` because there are no shares left to seize. `Unit.confiscate(holder, ...)` has the same problem.
- `fresh` is unfrozen and transfers the full amount onward, or stakes it again, or redeems it through `Minter2`.

No signer participates: `redeem` on the vault is a plain ERC-4626 call and does not pass through `Minter2`, so `SIGNER_ROLE` cannot gate it. Restricting the `receiver` does not close the path either, because a contract holder can redeem to itself and forward the UNIT in the same transaction.

Two points frame the severity honestly. First, the same window exists for liquid UNIT: a holder can `UNIT.transfer(fresh, all)` ahead of a freeze just as easily, so the vault is not the only exposure. It matters because a staked position is the one the operator can

otherwise seize cleanly through `StakedUnit.confiscate`, and because a vault is the natural place to add an exit delay without touching the stablecoin's transferability. Second, TRON has no priority-fee auction, but pending transactions are broadcast to every full node before inclusion and the ordering within a block is decided by the producing Super Representative, so a watched administrator address can be raced by broadcasting earlier or with a cooperating block producer. The condition is realistic for a holder who expects enforcement.

This finding is consistent with the earlier investigation showing that an already-frozen account cannot escape through the vault. That investigation covered the state after the freeze lands; this finding covers the ordering race before it does.

Proof of Concept

`test_M2_stakerFrontRunsFreezeByRedeemingToFreshAddress` in `test/AuditPoCs.t.sol` runs the sequence on the production `Unit`, `StakedUnit` and `Minter2` contracts using the existing fixture and helpers (`_mintFor`, `sUNIT`, `UNIT`, `admin`, `userA`, `userB`, `helper`). It also shows the equivalent window for a liquid holder.

```
function test_M2_stakerFrontRunsFreezeByRedeemingToFreshAddress() public {
    usdd.mint(address(minter2), 1e12); // M-1 seed
    _mintFor(userA, 100e6, true); // userA holds 100 sUNIT
    address fresh = makeAddr("fresh");

    // 1. Admin broadcasts setFrozen(userA). 2. userA's redeem is ordered first: no
    guard fires.
    vm.prank(userA);
    sUNIT.redeem(100e6, fresh, userA);

    // 3. The admin transaction lands on an empty account; the vault-level seizure
    has nothing to burn.
    vm.startPrank(admin);
    UNIT.setFrozen(userA, true);
    vm.expectRevert();
    sUNIT.confiscate(userA, admin, 100e6);
    vm.stopPrank();
}
```

```

// 4. The fresh address is unfrozen and moves the entire position freely.
assertTrue(UNIT.isFrozen(userA));
assertEq(sUNIT.balanceOf(userA), 0);
assertEq(UNIT.balanceOf(userA), 0);
assertEq(UNIT.balanceOf(fresh), 100e6);
vm.prank(fresh);
UNIT.transfer(userB, 100e6);
assertEq(UNIT.balanceOf(userB), 100e6);

// Same window for a liquid holder: a plain transfer ahead of the freeze.
_mintFor(helper, 50e6, false);
address fresh2 = makeAddr("fresh2");
vm.prank(helper);
UNIT.transfer(fresh2, 50e6);
vm.prank(admin);
UNIT.setFrozen(helper, true);
assertEq(UNIT.balanceOf(helper), 0);
assertEq(UNIT.balanceOf(fresh2), 50e6);
}

```

```
forge test --offline --match-path test/AuditPoCs.t.sol --match-test M2_ -vv
```

The test passes: the original account ends frozen and empty while an unfrozen address controls and transfers the full 100e6.

Impact

A targeted holder evades the protocol's freeze and confiscation control for the entire staked balance whenever it observes the administrative transaction and wins ordering against it. The administrator can freeze the next address, but the holder can repeat the move, and the protocol can never make the first seizure atomic. No other user's funds are at risk and nothing is minted or drained from backing. Severity is Medium because an explicit security and compliance control can be bypassed by an unprivileged actor under a realistic timing condition, and the control is the protocol's only enforcement mechanism against a sanctioned or compromised account.

Recommendation

Give the administrator a window in which a staked position cannot leave the vault. Add a two-step exit to `StakedUnit`: the owner announces the exit, the vault emits an event, and `_withdraw` only proceeds after a delay, re-evaluating the freeze flags at that later time. A freeze applied during the delay then blocks the exit, and `StakedUnit.confiscate` can still burn the shares because they never left the owner.

```
contract StakedUnit is ERC4626 {
    using SafeERC20 for IERC20;

+   uint64 public constant EXIT_DELAY = 1 days;
+   mapping(address => uint64) public exitReadyAt;
+
    event Confiscated(address indexed from, address indexed to, uint256 shares);
+   event ExitRequested(address indexed owner, uint64 readyAt);

    error NonTransferable();
    error AccountFrozen();
    error Unauthorized();
    error VaultInsolvent();
+   error ExitPending();

+   function requestExit() external {
+       uint64 readyAt = uint64(block.timestamp) + EXIT_DELAY;
+       exitReadyAt[msg.sender] = readyAt;
+       emit ExitRequested(msg.sender, readyAt);
+   }
+
+   function maxWithdraw(address owner) public view override returns (uint256) {
+       if (exitReadyAt[owner] == 0 || block.timestamp < exitReadyAt[owner]) return
0;
+       return _isSolvent() ? super.maxWithdraw(owner) : 0;
+   }
+
+   function maxRedeem(address owner) public view override returns (uint256) {
+       if (exitReadyAt[owner] == 0 || block.timestamp < exitReadyAt[owner]) return
0;
```

```

+         return _isSolvent() ? super.maxRedeem(owner) : 0;
+     }

    function _withdraw(address caller, address receiver, address owner, uint256
assets, uint256 shares)
        internal
        override
    {
        if (!_isSolvent()) revert VaultInsolvent();
+       if (exitReadyAt[owner] == 0 || block.timestamp < exitReadyAt[owner]) revert
ExitPending();
        Unit unit = Unit(address(asset()));
        if (unit.isFrozen(caller) || unit.isFrozen(receiver) ||
unit.isFrozen(owner)) revert AccountFrozen();
+       delete exitReadyAt[owner];
        super._withdraw(caller, receiver, owner, assets, shares);
    }

```

`Minter2.redeem` with `unstake == true` calls `stakedUnit.withdraw`, so the signing service must require a matured `requestExit` before issuing that permit, or the redeem-and-unstake path should be documented as two steps. Monitoring should alert on every `ExitRequested` event so the compliance process has the full delay to act.

If immediate exits must remain, treat the limitation as an operational one: submit freeze and confiscation transactions through a private path (a cooperating Super Representative or a relay that does not gossip the transaction before inclusion), batch `setFrozen`, `StakedUnit.confiscate` and `Unit.confiscate` into a single administrative transaction so that the seizure is atomic once it lands, and document that a publicly broadcast freeze cannot guarantee seizure. Note that only the delayed-exit design closes the staked leg; liquid UNIT keeps the same window unless transfers themselves are delayed, which is a product decision for a stablecoin rather than a code fix.

Status

Resolved

Low

[L-01] Unit#renounceRole - DEFAULT_ADMIN_ROLE can be renounced or revoked in a single step on Unit and Minter2

Description

Both `Unit` and `Minter2` inherit `OpenZeppelin AccessControl` without modification. The sole holder of `DEFAULT_ADMIN_ROLE` can call `renounceRole(DEFAULT_ADMIN_ROLE, self)` or `revokeRole(DEFAULT_ADMIN_ROLE, self)` in a single transaction, with no confirmation step, no time delay, and no check that a successor admin exists. The third contract in the system, `CumulativeMerkleDrop`, already treats the equivalent `Ownable` path as unacceptable and reverts on `renounceOwnership`, so the two `AccessControl` contracts are inconsistent with the protection the codebase has already adopted elsewhere.

Vulnerability Details

Both contracts pull in `AccessControl` as-is, at `src/Unit.sol:7` and `src/Minter2.sol:47`:

```
contract Unit is ERC20, AccessControl {
```

```
contract Minter2 is AccessControl, EIP712, Nonces {
```

Neither contract overrides `renounceRole` or `revokeRole`. OpenZeppelin's `renounceRole` only requires `callerConfirmation == msg.sender`, and `revokeRole` only requires the caller to hold the role's admin role, which for `DEFAULT_ADMIN_ROLE` is `DEFAULT_ADMIN_ROLE` itself. A single admin can therefore remove itself with one call.

By contrast, `CumulativeMerkleDrop` explicitly blocks the analogous foot-gun at `src/CumulativeMerkleDrop.sol:26-28`:

```
function renounceOwnership() public view override onlyOwner {  
    revert CannotRenounceOwnership();  
}
```

What is lost if the last admin is removed:

- On Unit: `grantRole(MINTER_ROLE, ...)` becomes impossible, so a redeployed or replacement Minter2 can never be authorised to mint and new UNIT supply ends permanently. `setFrozen` and `confiscate` are both `onlyRole(DEFAULT_ADMIN_ROLE)`, so no account can ever be frozen or unfrozen again and every existing freeze becomes permanent. `StakedUnit.confiscate` also checks the caller against `Unit.DEFAULT_ADMIN_ROLE`, so vault-level confiscation is lost as well.
- On Minter2: the yield sweep `withdraw`, the escape hatch `executeCall`, and all role management (`SIGNER_ROLE`, `DISTRIBUTOR_ROLE`, `KEEPER_ROLE`) are lost while the entire protocol backing in USDD and jUSDD remains inside the contract. `mint` and `redeem` continue to work because they only depend on `SIGNER_ROLE`, but the protocol can never recover surplus, rotate a compromised signer, or react to a change in the PSM or JustLend integration.

Proof of Concept

Adapted from `test_L1_adminRenounceStrandsUnitAndMinter2` in `test/AuditPoCs.t.sol`. The `admin` address is the sole `DEFAULT_ADMIN_ROLE` holder on both contracts after `setUp`.

```
function test_L1_adminRenounceStrandsUnitAndMinter2() public {
    bytes32 ADMIN = 0x00;
    bytes32 MINTER = UNIT.MINTER_ROLE();

    // One call each, no confirmation, no delay, no successor check.
    vm.startPrank(admin);
    UNIT.renounceRole(ADMIN, admin);
    minter2.renounceRole(ADMIN, admin);
    vm.stopPrank();

    vm.startPrank(admin);
    // Unit: nobody can ever grant MINTER_ROLE again, so a redeployed Minter2 could
    never mint.

    vm.expectRevert(abi.encodeWithSelector(IAccessControl.AccessControlUnauthorizedAccount.selector, admin, ADMIN));
    UNIT.grantRole(MINTER, userA);

    // Unit: freeze / unfreeze / confiscate are permanently unavailable.
```

```

vm.expectRevert(abi.encodeWithSelector(IAccessControl.AccessControlUnauthorizedAccount.selector, admin, ADMIN));
    UNIT.setFrozen(userA, true);

    // Minter2: the yield sweep and executeCall are gone while all backing stays inside the contract.

vm.expectRevert(abi.encodeWithSelector(IAccessControl.AccessControlUnauthorizedAccount.selector, admin, ADMIN));
    minter2.withdraw(IERC20(jUSDD_ADDR), admin, 1);
    vm.stopPrank();
}

```

```
forge test --match-path test/AuditPoCs.t.sol --match-test L1 -vv
```

Impact

A single mistaken or malicious transaction by the admin key permanently removes every privileged capability from Unit and Minter2. On Unit this ends new supply forever once the current Minter2 is retired and makes every freeze irreversible. On Minter2 it strands the accumulated yield surplus and removes every operational lever, including signer rotation, while user mints and redeems continue. The precondition is an operator error rather than an external attacker, and no funds are stolen, but the consequence cannot be undone and the codebase already treats the equivalent Ownable path as a defect worth blocking.

Recommendation

Preferred: replace AccessControl with OpenZeppelin's AccessControlDefaultAdminRules, which enforces a single admin, a two-step transfer with a configurable delay, and only allows renouncing DEFAULT_ADMIN_ROLE after a scheduled transfer to address(0) has matured.

```

-import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
+import {AccessControlDefaultAdminRules} from

```

```
"@openzeppelin/contracts/access/extensions/AccessControlDefaultAdminRules.sol";

- contract Unit is ERC20, AccessControl {
+ contract Unit is ERC20, AccessControlDefaultAdminRules {
    ...
-     constructor(address admin_) ERC20("Unit USD", "unitUSD") {
-         if (admin_ == address(0)) revert ZeroAddress();
-         _grantRole(DEFAULT_ADMIN_ROLE, admin_);
-     }
+     constructor(address admin_)
+         ERC20("Unit USD", "unitUSD")
+         AccessControlDefaultAdminRules(2 days, admin_)
+     {}
```

Apply the same change to `Minter2`, keeping the explicit `KEEPER_ROLE` grant in its constructor. Note that `AccessControlDefaultAdminRules` also blocks `grantRole(DEFAULT_ADMIN_ROLE, ...)` and `revokeRole(DEFAULT_ADMIN_ROLE, ...)`, forcing all admin changes through `beginDefaultAdminTransfer` and `acceptDefaultAdminTransfer`.

Minimal alternative if the extension is not wanted: keep `AccessControl`, refuse to drop the admin role through the generic paths, and provide an explicit two-step handover.

```
error CannotDropAdmin();
error NotPendingAdmin();

address public pendingAdmin;

function renounceRole(bytes32 role, address callerConfirmation) public override {
    if (role == DEFAULT_ADMIN_ROLE) revert CannotDropAdmin();
    super.renounceRole(role, callerConfirmation);
}

function revokeRole(bytes32 role, address account) public override {
    if (role == DEFAULT_ADMIN_ROLE) revert CannotDropAdmin();
    super.revokeRole(role, account);
}
```

```

function transferAdmin(address newAdmin) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newAdmin == address(0)) revert ZeroAddress();
    pendingAdmin = newAdmin;
}

function acceptAdmin() external {
    if (msg.sender != pendingAdmin) revert NotPendingAdmin();
    address previous = getRoleMember(DEFAULT_ADMIN_ROLE, 0); // requires
    AccessControlEnumerable
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    _revokeRole(DEFAULT_ADMIN_ROLE, previous);
    pendingAdmin = address(0);
}

```

Whichever variant is chosen, apply it identically to `Unit` and `Minter2` so all three contracts share the same admin-safety posture.

Status

Resolved

[L-02] Unit#confiscate - No guard for protocol-owned addresses; confiscating from the vault locks every staker

Description

`Unit.confiscate(from, to, value)` is a raw `_transfer` executed with admin privilege. Because `Unit._update` skips the freeze checks whenever `msg.sender` holds `DEFAULT_ADMIN_ROLE`, the call bypasses the freeze and accepts any `from` address, including the protocol's own contracts. If `from` is the `StakedUnit` vault, the vault's `totalAssets()` falls below its `totalSupply()`, `_isSolvent()` returns false, and every `deposit`, `mint`, `withdraw` and `redeem` reverts for every staker until the admin returns the shortfall. `setFrozen` has the same shape: applied to `StakedUnit`, `Minter2` or `CumulativeMerkleDrop` it halts every transfer that passes through that contract. The correct instrument for a misbehaving staker is `StakedUnit.confiscate(from, to, shares)`, which burns the target's shares and keeps the vault solvent, but nothing in the `Unit` interface prevents an operator from reaching for the wrong one.

Vulnerability Details

`confiscate` at `src/Unit.sol:51-54` performs no validation on `from`:

```
function confiscate(address from, address to, uint256 value) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    _transfer(from, to, value);
    emit Confiscated(from, to, value);
}
```

The freeze hook at `src/Unit.sol:56-62` is deliberately disabled for admin callers so that confiscation from a frozen account works, which also means no check of any kind runs on the `from` address:

```
function _update(address from, address to, uint256 value) internal override {
    if (!hasRole(DEFAULT_ADMIN_ROLE, msg.sender)) {
        if (from != address(0) && isFrozen[from]) revert AccountFrozen();
        if (to != address(0) && isFrozen[to]) revert AccountFrozen();
    }
    super._update(from, to, value);
}
```

`StakedUnit` is a strict 1:1 vault whose every entry point is gated on solvency, at `src/StakedUnit.sol:30-48`:

```
function _isSolvent() internal view returns (bool) {
    return totalAssets() >= totalSupply();
}

function maxDeposit(address receiver) public view override returns (uint256) {
    return _isSolvent() ? super.maxDeposit(receiver) : 0;
}

function maxMint(address receiver) public view override returns (uint256) {
    return _isSolvent() ? super.maxMint(receiver) : 0;
}

function maxWithdraw(address owner) public view override returns (uint256) {
    return _isSolvent() ? super.maxWithdraw(owner) : 0;
}

function maxRedeem(address owner) public view override returns (uint256) {
    return _isSolvent() ? super.maxRedeem(owner) : 0;
}
```

`_deposit` and `_withdraw` at `src/StakedUnit.sol:60-75` additionally revert `VaultInsolvent()` directly. Pulling even one unit of UNIT out of the vault through `Unit.confiscate` therefore flips every `max*` view to zero and makes every state-changing vault call revert. Because `Minter2.mint(..., stake = true)` calls `stakedUnit.deposit` and `Minter2.redeem(..., unstake = true)` calls `stakedUnit.withdraw`, both of `Minter2`'s staking paths fail as well.

Related consequences of the same missing guard:

- `Unit.confiscate(address(drop), ...)` removes UNIT from `CumulativeMerkleDrop`, so every later `claim` reverts inside `safeTransfer` once the remaining balance is exhausted.
- `Unit.setFrozen(address(sUNIT), true)` blocks every vault deposit and withdrawal, because `Unit._update` now rejects the vault as `from` or `to` for every non-admin caller.
- `Unit.setFrozen(address(minter2), true)` blocks every `mint` and `redeem`, since `Minter2`

receives, burns and forwards UNIT on those paths.

The safe alternative already exists at `src/StakedUnit.sol:50-58`: `StakedUnit.confiscate` burns the target's shares and transfers the matching UNIT out, so `totalAssets()` and `totalSupply()` fall together and the vault stays solvent.

Proof of Concept

Adapted from `test_L2_confiscateFromVaultLocksAllStakers` and `test_L2_confiscateFromRewardsDropBreaksLaterClaims` in `test/AuditPoCs.t.sol`.

```
function test_L2_confiscateFromVaultLocksAllStakers() public {
    usdd.mint(address(minter2), 1e12); // seed so the M-1 backing check passes
    _mintFor(userA, 100e6, true);      // userA stakes 100 UNIT
    _mintFor(userB, 100e6, true);      // userB stakes 100 UNIT

    // Admin intends to confiscate userA's stake but uses Unit.confiscate on the
    vault address.
    vm.prank(admin);
    UNIT.confiscate(address(sUNIT), admin, 100e6);

    // The vault is now insolvent and EVERY staker is locked, including the innocent
    userB.
    assertEq(sUNIT.maxWithdraw(userB), 0);
    vm.prank(userB);
    vm.expectRevert();
    sUNIT.redeem(100e6, userB, userB);

    // Minter2's unstake path is dead too.
    vm.startPrank(userB);
    sUNIT.approve(address(minter2), 100e6);
    bytes memory sig = _sig(minter2.REDEEM_TYPEHASH(), userB, 100e6, true,
    block.timestamp + 1 hours);
    vm.expectRevert();
    minter2.redeem(100e6, true, 0, block.timestamp + 1 hours, sig);
    vm.stopPrank();
}
```

```
function test_L2_confiscateFromRewardsDropBreaksLaterClaims() public {
    vm.prank(admin);
    UNIT.mint(address(drop), 100e6);
    vm.prank(admin);
    UNIT.confiscate(address(drop), admin, 100e6);
    assertEq(UNIT.balanceOf(address(drop)), 0); // every outstanding claim now
    reverts on transfer
}
```

```
forge test --match-path test/AuditPoCs.t.sol --match-test L2 -vv
```

Impact

A single `Unit.confiscate` call with the vault address as `from` produces a protocol-wide staking outage: every staker, not just the intended target, loses the ability to deposit, withdraw or redeem, and Minter2's mint-and-stake and redeem-and-unstake paths revert. The same call aimed at `CumulativeMerkleDrop` makes outstanding reward claims fail, and `setFrozen` aimed at any protocol contract halts the flows through it. The admin is trusted and the state is recoverable by transferring the shortfall back, but the interface offers no protection against an ordinary-looking call producing this result, and users are locked out until the operator notices and reverses it.

Recommendation

Maintain an admin-managed set of protected addresses in `Unit` (the staking vault, the active minter, the rewards drop) and refuse to `confiscate` from or `setFrozen` on any of them. Vault-level confiscation then has exactly one path, `StakedUnit.confiscate`, which preserves solvency by construction.

```
contract Unit is ERC20, AccessControl {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");

    mapping(address => bool) public isFrozen;
    + mapping(address => bool) public isProtected;

    event Confiscated(address indexed from, address indexed to, uint256 value);
```

```

    event Frozen(address indexed account);
    event Unfrozen(address indexed account);
+   event ProtectedSet(address indexed account, bool protected);

    error ZeroAddress();
    error AccountFrozen();
+   error ProtectedAddress();

+   function setProtected(address account, bool protected) external
onlyRole(DEFAULT_ADMIN_ROLE) {
+       if (account == address(0)) revert ZeroAddress();
+       isProtected[account] = protected;
+       emit ProtectedSet(account, protected);
+   }

    function setFrozen(address account, bool frozen) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    if (account == address(0)) revert ZeroAddress();
+   if (isProtected[account]) revert ProtectedAddress();
    isFrozen[account] = frozen;
    ...
}

    function confiscate(address from, address to, uint256 value) external
onlyRole(DEFAULT_ADMIN_ROLE) {
+   if (isProtected[from]) revert ProtectedAddress();
    _transfer(from, to, value);
    emit Confiscated(from, to, value);
}

```

Register `StakedUnit`, `Minter2` and `CumulativeMerkleDrop` via `setProtected` as part of the deployment script, and update the set whenever a protocol contract is replaced. If a protected address ever needs to be frozen or drained in an emergency, the admin can first call `setProtected(account, false)`, which makes the destructive intent explicit in the transaction history.

Status

Resolved



Hashlock Pty Ltd

[L-03] CumulativeMerkleDrop#setMerkleRoot - Accepts any root including zero and silently disables claims

Description

CumulativeMerkleDrop.setMerkleRoot stores whatever value the owner passes without validation. Setting the root to bytes32(0) disables every claim: callers who present the previous root fail the Merkle root changed check, and callers who present the zero root fail Invalid proof because no leaf can hash to zero. Any other incorrect root has the same effect. The state is owner-recoverable, but the contract exposes no explicit pause flag or dedicated event, so integrators and users cannot distinguish an intentional pause from a misconfigured root.

Vulnerability Details

The setter at src/CumulativeMerkleDrop.sol:30-33 is a bare assignment:

```
function setMerkleRoot(bytes32 newRoot) external onlyOwner {
    merkleRoot = newRoot;
    emit MerkleRootUpdated(newRoot);
}
```

Every claim is gated on the stored root twice, at src/CumulativeMerkleDrop.sol:41-44:

```
require(merkleRoot == expectedMerkleRoot, "Merkle root changed");

bytes32 leaf = keccak256(abi.encodePacked(account, cumulativeAmount));
require(MerkleProof.verify(merkleProof, merkleRoot, leaf), "Invalid proof");
```

With merkleRoot == bytes32(0):

- A claimant using a proof generated against the previous, correct root passes a non-zero expectedMerkleRoot and fails the first require.
- A claimant who notices the change and passes expectedMerkleRoot = bytes32(0) reaches MerkleProof.verify, which folds the proof into a keccak256 chain that cannot produce zero for any real leaf, so the second require fails.

The result is that all reward distribution stops. The MerkleRootUpdated event fires with a

zero argument, which is indistinguishable in shape from a legitimate root rotation, so off-chain monitoring has no clear signal that the drop has been disabled. The constructor at `src/CumulativeMerkleDrop.sol:21-24` also accepts a zero initial root, which is a reasonable "not yet configured" state, but the setter should not allow a return to it once live UNIT rewards are held.

Proof of Concept

Adapted from `test_L3_zeroRootDisablesAllClaims` in `test/AuditPoCs.t.sol`. A single-leaf tree is used so the root equals the leaf and the proof is empty.

```
function test_L3_zeroRootDisablesAllClaims() public {
    vm.prank(admin);
    UNIT.mint(address(drop), 100e6);

    bytes32 leaf = keccak256(abi.encodePacked(userA, uint256(50e6)));
    vm.prank(admin);
    drop.setMerkleRoot(leaf); // single-leaf tree, empty proof
    drop.claim(userA, 50e6, leaf, new bytes32[](0));
    assertEq(UNIT.balanceOf(userA), 50e6); // claims work with a valid root

    vm.prank(admin);
    drop.setMerkleRoot(bytes32(0)); // accepted silently

    // Old root: rejected by the first require.
    vm.expectRevert(bytes("Merkle root changed"));
    drop.claim(userA, 60e6, leaf, new bytes32[](0));

    // Zero root: rejected by the proof check, since no leaf hashes to zero.
    vm.expectRevert(bytes("Invalid proof"));
    drop.claim(userA, 60e6, bytes32(0), new bytes32[](0));
}
```

```
forge test --match-path test/AuditPoCs.t.sol --match-test L3 -vv
```

Impact

Every reward claim reverts until the owner sets a correct root. No funds are lost and the owner can recover the state at any time, but `setMerkleRoot` is the single control over a contract that holds live UNIT rewards, and one wrong argument halts distribution for all recipients with no distinct on-chain signal. Because the same setter is also the only way to pause the drop, operators are pushed toward using an invalid root as a pause mechanism, which normalises exactly the state that this finding describes.

Recommendation

Reject the zero root in the setter and, if pausing is a product requirement, add an explicit pause flag with its own events rather than overloading the root.

```
contract CumulativeMerkleDrop is Ownable2Step {
    using SafeERC20 for IERC20;

    IERC20 public immutable token;
    bytes32 public merkleRoot;
+   bool public paused;

    mapping(address => uint256) public cumulativeClaimed;

    event MerkleRootUpdated(bytes32 indexed newRoot);
    event Claimed(address indexed account, uint256 amount);
+   event Paused();
+   event Unpaused();

    error CannotRenounceOwnership();
+   error ZeroRoot();
+   error ClaimsPaused();

    function setMerkleRoot(bytes32 newRoot) external onlyOwner {
+       if (newRoot == bytes32(0)) revert ZeroRoot();
        merkleRoot = newRoot;
        emit MerkleRootUpdated(newRoot);
    }
}
```

```

+   function setPaused(bool paused_) external onlyOwner {
+       paused = paused_;
+       if (paused_) emit Paused();
+       else emit Unpaused();
+   }

    function claim(
        address account,
        uint256 cumulativeAmount,
        bytes32 expectedMerkleRoot,
        bytes32[] calldata merkleProof
    ) external {
+       if (paused) revert ClaimsPaused();
        require(merkleRoot == expectedMerkleRoot, "Merkle root changed");
        ...
    }

```

Optionally apply the same `ZeroRoot` check in the constructor if the deployment flow always provides a real root, and have the off-chain tooling verify that at least one known leaf proves against `newRoot` before submitting `setMerkleRoot`.

Status

Resolved

QA

[Q-01] Minter2 - External kill-switches and liquidity dependencies can halt mint or redeem

Description

Minter2 has no fallback for any of its three external integrations. Each of UsddPsm, TRON USDT and JustLend jUSDD carries owner-controlled switches or liquidity limits that, when flipped or exhausted, make mint or redeem revert. None of these is a defect in Minter2's code, and all live values were verified healthy on 2026-09-22, but each belongs in the operational runbook and in monitoring because Minter2 offers users no explanation and offers operators no bypass when one of them trips.

Vulnerability Details

The mint path pulls exactly assets of USDT from the caller and then sells exactly assets to the PSM, so it assumes that the amount received equals the amount requested and that the PSM will accept it:

```
// src/Minter2.sol:188-198
function _mintInternal(uint256 assets) private returns (uint256) {
    USDT.safeTransferFrom(msg.sender, address(this), assets);

    uint256 usddBefore = USDD.balanceOf(address(this));
    PSM.sellGem(address(this), assets);
    uint256 usddReceived = USDD.balanceOf(address(this)) - usddBefore;

    uint256 unitToMint = _depositToJustLend(usddReceived);
    if (assets > 0 && unitToMint == 0) revert OperationFailed();
    return unitToMint;
}
```

The dependencies, with their live values on 2026-09-22:

- UsddPsm (TBXW4hS5KYjjbJXDpnrPf4zhkLwrrpUjbyz), called through sellGem at src/Minter2.sol:192 and buyGem at src/Minter2.sol:234. sellEnabled and buyEnabled are both 1; setting either to 0 halts mint or redeem respectively. tin and tout are both 0;



a non-zero `tin` reduces the USDD Minter2 receives per USDT, which `_depositToJustLend` then rejects via the per-deposit rounding check (`src/Minter2.sol:214-217`) because the shortfall exceeds the tolerance, so `mint` reverts with `OperationFailed`. The PSM's debt ceiling caps total `sellGem` volume, and the USDT reserve held by the `gemJoin` (`TSUYvQ5tdd3DijCD1uGunGLpftHuSZ12sQ`) caps `buyGem`, so redeem availability depends on other PSM users' net flow.

- TRON USDT (`TR7NHqjeKQxGTCi8q8ZY4pL8otSzgjLj6t`). `paused` is 0; a pause blocks both paths at the first transfer. The owner-managed blacklist can include Minter2 itself, which would freeze all backing flows through the contract. `basisPointsRate` is 0; if the owner enables a transfer fee, `safeTransferFrom` at `src/Minter2.sol:189` credits Minter2 with `assets` minus the fee while `sellGem(assets)` at `src/Minter2.sol:192` pulls the full `assets`, so `mint` reverts on the USDT balance underflow inside the token. No user-supplied `assets` value can clear the check, so the fee is a full mint outage rather than a degraded one.

- JustLend `jUSDD` (`TKFRELGGorGgiayhwJTNNLqCNjFoLBh3Mnf`). A mint pause or supply cap on the market makes `jUSDD.mint` at `src/Minter2.sol:204` return a non-zero error code, which Minter2 correctly converts to `OperationFailed`, halting both `mint` and `claimAndDistributeRewards`. `redeemUnderlying` at `src/Minter2.sol:230` is bounded by the market's cash (about 397 million USDD today) and its utilisation; a large redemption or a utilisation spike makes `redeem` revert until liquidity returns. The supply APY is about 0.00085 percent, which is why the surplus discussed in the Medium finding grows so slowly.

Minter2 already validates the static integration wiring in its constructor (`src/Minter2.sol:85-94`), but it reads none of the dynamic switches, so a user or keeper sees only a generic `OperationFailed` or a raw token revert.

Proof of Concept

The test models the TRON USDT fee switch. `FeeTRONUSDT` mirrors `TetherToken` semantics: the sender is debited the full value, the recipient receives value minus fee, and `transfer` returns `false` while `transferFrom` returns `true`, matching the live quirk. It reuses the fixture and helpers from `test/AuditPoCs.t.sol`.

```

/// TRON USDT (TetherToken) with a non-zero basisPointsRate.
contract FeeTRONUSDT {
    mapping(address => uint256) public balanceOf;
    mapping(address => mapping(address => uint256)) public allowance;
    uint256 public basisPointsRate;
    uint256 public maximumFee;
    address constant FEE_COLLECTOR = address(0xFEE);

    function decimals() external pure returns (uint8) { return 6; }
    function setParams(uint256 bps, uint256 maxFee) external { basisPointsRate = bps;
maximumFee = maxFee; }
    function mint(address to, uint256 v) external { balanceOf[to] += v; }
    function approve(address s, uint256 v) external returns (bool) {
allowance[msg.sender][s] = v; return true; }

    function transfer(address to, uint256 v) external returns (bool) {
_move(msg.sender, to, v); return false; } // TRON quirk
    function transferFrom(address from, address to, uint256 v) external returns
(bool) {
        if (allowance[from][msg.sender] != type(uint256).max)
allowance[from][msg.sender] -= v;
        _move(from, to, v);
        return true;
    }

    function _move(address from, address to, uint256 v) internal {
        uint256 fee = (v * basisPointsRate) / 10000;
        if (fee > maximumFee) fee = maximumFee;
        balanceOf[from] -= v; // full value leaves the sender: this is what
reverts inside sellGem
        balanceOf[to] += v - fee;
        balanceOf[FEE_COLLECTOR] += fee;
    }
}

function test_Q2_usdtTransferFeeBreaksMint() public {
    usdd.mint(address(minter2), 1e12); // M-1 seed so the failure below is the fee,
not the backing invariant

```

```

    // Replace USDT with the fee-enabled model and re-establish the approvals
    Minter2's constructor made.
    vm.etch(USDT_ADDR, address(new FeeTRONUSDT()).code);
    FeeTRONUSDT fee = FeeTRONUSDT(USDT_ADDR);
    fee.setParams(10, type(uint256).max); // 10 bps = 0.1 %, a value the USDT owner
    can set at any time
    vm.startPrank(address(minter2));
    fee.approve(address(minter2), type(uint256).max);
    fee.approve(psm.gemJoin(), type(uint256).max);
    vm.stopPrank();

    fee.mint(userA, 100e6);
    vm.startPrank(userA);
    fee.approve(address(minter2), 100e6);
    bytes memory sig = _sig(minter2.MINT_TYPEHASH(), userA, 100e6, false,
    block.timestamp + 1 hours);
    // Minter2 receives 99.9 USDT, then sellGem(100 USDT) pulls the full 100: balance
    underflow inside USDT.
    vm.expectRevert();
    minter2.mint(100e6, false, 0, block.timestamp + 1 hours, sig);
    vm.stopPrank();

    assertEq(UNIT.totalSupply(), 0);
    assertEq(fee.balanceOf(userA), 100e6); // the user keeps their USDT; the protocol
    simply cannot mint
}

```

```
forge test --match-test test_Q2_usdtTransferFeeBreaksMint -vv
```

Read-only monitoring of the upstream switches through TronGrid. Each call is a `triggerconstantcontract` against the dependency; alert on any deviation from the values recorded on 2026-09-22.

```

API=https://api.trongrid.io/wallet/triggerconstantcontract
MINTER2=TYpL3Fiye4i44SAWUx4NCok2D6bPzSdFKH
call() {

```

```

curl -s -X POST "$API" -H 'Content-Type: application/json' \
  -d
  "{ \"owner_address\": \"$MINTER2\", \"contract_address\": \"$1\", \"function_selector\": \"$2\", \"visible\": true }" \
  | jq -r '.constant_result[0]'
}
echo "UsddPsm: expect 1 / 1 / 0 / 0"
call TBXW4hS5KYjjbJXDpnrPf4zhkLwrpUjbyz 'sellEnabled()'
call TBXW4hS5KYjjbJXDpnrPf4zhkLwrpUjbyz 'buyEnabled()'
call TBXW4hS5KYjjbJXDpnrPf4zhkLwrpUjbyz 'tin()'
call TBXW4hS5KYjjbJXDpnrPf4zhkLwrpUjbyz 'tout()'
echo "TRON USDT: expect 0 / 0; also poll isBlackListed(address) with Minter2's
address as the ABI-encoded parameter"
call TR7NHqjeKQxGTCi8q8ZY4pL8otSzgJLj6t 'paused()'
call TR7NHqjeKQxGTCi8q8ZY4pL8otSzgJLj6t 'basisPointsRate()'
echo "jUSDD: cash available to redeemUnderlying (about 397e24 today) and the current
exchange rate"
call TKFRELGGorGiayhwJTNNLqCNjFoLBh3Mnf 'getCash()'
call TKFRELGGorGiayhwJTNNLqCNjFoLBh3Mnf 'exchangeRateStored()'

```

Impact

While any one switch is flipped, mint, redeem, or both are unavailable for every user, and the reward pipeline (`claimAndDistributeRewards`) stops whenever the jUSDD market refuses deposits. No backing is lost: USDD and jUSDD stay in Minter2, and a USDT blacklisting of Minter2 would strand backing only until the listing is lifted. Users see an opaque `OperationFailed` or raw token revert with no signal about which dependency failed, and operators have no on-chain switch of their own to pause cleanly ahead of an upstream change.

Recommendation

Operational controls first: add the parameters above to monitoring with alerts on any change, and write runbook entries that name which Minter2 function each switch disables and what the recovery step is (wait for the PSM or USDT owner, migrate backing via `withdraw`, or redeploy).

In code, make `_mintInternal` measure what it actually received rather than trusting `assets`, so a future USDT fee produces a clear, deterministic outcome instead of a token-level underflow. The signed `assets` value still bounds what the user pays, and `minUnitOut` still protects the user against receiving fewer UNIT than they expect:

```
error InvalidIntegration();
    error InsufficientOutput();
+   error TransferFeeActive();

function _mintInternal(uint256 assets) private returns (uint256) {
-   USDT.safeTransferFrom(msg.sender, address(this), assets);
+   uint256 usdtBefore = USDT.balanceOf(address(this));
+   USDT.safeTransferFrom(msg.sender, address(this), assets);
+   uint256 usdtReceived = USDT.balanceOf(address(this)) - usdtBefore;
+   // A transfer fee on USDT is a protocol-level event, not something to
    silently absorb.
+   if (usdtReceived != assets) revert TransferFeeActive();

    uint256 usddBefore = USDD.balanceOf(address(this));
-   PSM.sellGem(address(this), assets);
+   PSM.sellGem(address(this), usdtReceived);
    uint256 usddReceived = USDD.balanceOf(address(this)) - usddBefore;
```

If the team prefers to keep minting under a fee rather than halting, replace the revert with selling `usdtReceived` and let `minUnitOut` guard the user. Either way the balance-delta approach needs no knowledge of TetherToken's ABI. Reading `paused()`, `basisPointsRate()` and `isBlackListed(address)` directly is also possible, but couples Minter2 to TRON USDT's specific interface and is better left to off-chain monitoring.

Status

Acknowledged

[Q-02] Unit#_update - DEFAULT_ADMIN_ROLE holders bypass freeze checks; granting the role to a protocol contract would open a freeze escape

Description

Unit._update (src/Unit.sol:56-62) skips both isFrozen checks whenever msg.sender holds DEFAULT_ADMIN_ROLE. This is by design: it is what allows Unit.confiscate to move tokens out of a frozen account. The hypothesised freeze escape through StakedUnit shares was investigated and is not reproducible in the current code; every path reverts with AccountFrozen or NonTransferable, and the deployed sUNIT exposes the same guards. That guarantee, however, rests entirely on DEFAULT_ADMIN_ROLE never being granted to a protocol contract. If Minter2 ever held the role, its mint and redeem paths would silently stop enforcing the freeze for every user, because the only freeze check on those paths lives inside Unit._update.

Vulnerability Details

The bypass is keyed on the caller's role, not on the operation being performed:

```
// src/Unit.sol:56-62
function _update(address from, address to, uint256 value) internal override {
    if (!hasRole(DEFAULT_ADMIN_ROLE, msg.sender)) {
        if (from != address(0) && isFrozen[from]) revert AccountFrozen();
        if (to != address(0) && isFrozen[to]) revert AccountFrozen();
    }
    super._update(from, to, value);
}
```

StakedUnit performs its own freeze checks, so the vault legs stay protected regardless of who holds the admin role:

```
// src/StakedUnit.sol:60-75
function _deposit(address caller, address receiver, uint256 assets, uint256 shares)
internal override {
    if (!_isSolvent()) revert VaultInsolvent();
    Unit unit = Unit(address(asset()));
    if (unit.isFrozen(caller) || unit.isFrozen(receiver)) revert AccountFrozen();
    super._deposit(caller, receiver, assets, shares);
}
```

```

}

function _withdraw(address caller, address receiver, address owner, uint256 assets,
uint256 shares)
    internal
    override
{
    if (!_isSolvent()) revert VaultInsolvent();
    Unit unit = Unit(address(asset()));
    if (unit.isFrozen(caller) || unit.isFrozen(receiver) || unit.isFrozen(owner))
revert AccountFrozen();
    super._withdraw(caller, receiver, owner, assets, shares);
}

```

The unstaked Minter2 paths have no equivalent check of their own and rely entirely on `Unit._update`:

- `mint` with `stake == false` calls `UNIT.mint(msg.sender, unitToMint)` at `src/Minter2.sol:126`. With Minter2 as `msg.sender` inside `_update`, an admin-role Minter2 would credit fresh UNIT to a frozen account.
- `redeem` with `unstake == false` calls `UNIT.burnFrom(msg.sender, assets)` at `src/Minter2.sol:145`, then pays USDT to `msg.sender` at `src/Minter2.sol:236`. With Minter2 holding the admin role, a frozen account burns its UNIT and receives USDT, which is a complete exit from the freeze.
- `mint` with `stake == true` and `redeem` with `unstake == true` route through `StakedUnit._deposit` / `_withdraw`, whose own `isFrozen` checks still fire, so the staked legs remain closed.

Granting the role to `StakedUnit` instead relaxes only the UNIT transfer leg inside the vault. `_deposit` and `_withdraw` re-check caller, receiver and owner before calling `super`, so the ordinary vault paths stay closed. The one path that opens is `StakedUnit.confiscate` (`src/StakedUnit.sol:50-58`): after `_burn(from, shares)` it calls `IERC20(asset()).safeTransfer(to, shares)`, and with `StakedUnit` as an admin caller `Unit._update` would no longer reject a frozen `to`.

The test suite and the deployment grant `Minter2` and `StakedUnit` only `MINTER_ROLE`. `test_frozenAccountCannotEscapeThroughVault` in `test/AuditPoCs.t.sol` exercises every deposit, transfer, withdraw, delegate and `Minter2` path for a frozen account under that configuration and asserts that both the liquid and staked balances are unchanged.

Proof of Concept

The test first confirms that the current configuration blocks a frozen account's redeem, then grants `DEFAULT_ADMIN_ROLE` to `Minter2` and shows the same redeem succeeding. It reuses the fixture and helpers from `test/AuditPoCs.t.sol`.

```
function test_Q3_adminRoleOnMinter2DisablesFreezeOnRedeem() public {
    usdd.mint(address(minter2), 1e12); // M-1 seed
    _mintFor(userA, 100e6, false);    // 100 UNIT liquid
    vm.prank(admin);
    UNIT.setFrozen(userA, true);

    // Current configuration (Minter2 holds MINTER_ROLE only): the frozen account
    cannot exit.
    vm.startPrank(userA);
    UNIT.approve(address(minter2), 100e6);
    bytes memory sig = _sig(minter2.REDEEM_TYPEHASH(), userA, 100e6, false,
block.timestamp + 1 hours);
    vm.expectRevert(Unit.AccountFrozen.selector);
    minter2.redeem(100e6, false, 0, block.timestamp + 1 hours, sig);
    vm.stopPrank();

    // Misconfiguration: Minter2 is granted DEFAULT_ADMIN_ROLE on Unit.
    vm.prank(admin);
    UNIT.grantRole(UNIT.DEFAULT_ADMIN_ROLE(), address(minter2));

    // The identical call now succeeds. Unit._update sees an admin msg.sender and
    skips the isFrozen checks.
    uint256 usdtBefore = usdt.balanceOf(userA);
    vm.startPrank(userA);
    sig = _sig(minter2.REDEEM_TYPEHASH(), userA, 100e6, false, block.timestamp + 1
hours); // reverted call consumed no nonce
    minter2.redeem(100e6, false, 0, block.timestamp + 1 hours, sig);
```

```

vm.stopPrank();

// The frozen account converted its entire UNIT position to USDT while still
// flagged as frozen.
assertTrue(UNIT.isFrozen(userA));
assertEq(UNIT.balanceOf(userA), 0);
assertEq(usdt.balanceOf(userA) - usdtBefore, 100e6);
}

```

```

forge test --match-test test_Q3_adminRoleOnMinter2DisablesFreezeOnRedeem -vv
forge test --match-test test_frozenAccountCannotEscapeThroughVault -vv # proves the
current configuration is safe

```

Impact

None in the current configuration. If a future operator grants `DEFAULT_ADMIN_ROLE` to `Minter2` (for example to let it call `withdraw`-style maintenance on `Unit`, or by mistake when rotating roles), every frozen account regains the ability to mint `UNIT` to itself and to redeem its full `UNIT` balance for `USDT` through `Minter2`, with no event or revert to signal that the freeze has stopped applying. The compliance guarantee that `setFrozen` and `confiscate` are meant to provide would be void for the unstaked paths while appearing intact on-chain.

Recommendation

Keep `DEFAULT_ADMIN_ROLE` off every protocol contract and record this as an explicit invariant in the deployment runbook and in a test that asserts `!UNIT.hasRole(DEFAULT_ADMIN_ROLE, address(minter2))` and the same for `stakedUnit` after deployment.

To remove the dependency on role hygiene, scope the freeze bypass to the single operation that needs it instead of to every admin-role caller. A private flag set only inside `confiscate` keeps `confiscate` working on frozen accounts while every other transfer, including an admin's ordinary `transfer`, respects the freeze:

```

contract Unit is ERC20, AccessControl {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");

    mapping(address => bool) public isFrozen;
+   bool private _confiscating;

    ...

    function confiscate(address from, address to, uint256 value) external
onlyRole(DEFAULT_ADMIN_ROLE) {
-       _transfer(from, to, value);
+       _confiscating = true;
+       _transfer(from, to, value);
+       _confiscating = false;
        emit Confiscated(from, to, value);
    }

    function _update(address from, address to, uint256 value) internal override {
-       if (!hasRole(DEFAULT_ADMIN_ROLE, msg.sender)) {
+       if (!_confiscating) {
            if (from != address(0) && isFrozen[from]) revert AccountFrozen();
            if (to != address(0) && isFrozen[to]) revert AccountFrozen();
        }
        super._update(from, to, value);
    }
}

```

With this change, granting `DEFAULT_ADMIN_ROLE` to `Minter2` or `StakedUnit` no longer weakens the freeze, and `Unit.confiscate` remains the one sanctioned way to move tokens out of a frozen account. If the team relies on admin-initiated plain transfers to frozen accounts today, route those through `confiscate` instead so the exception stays explicit and evented.

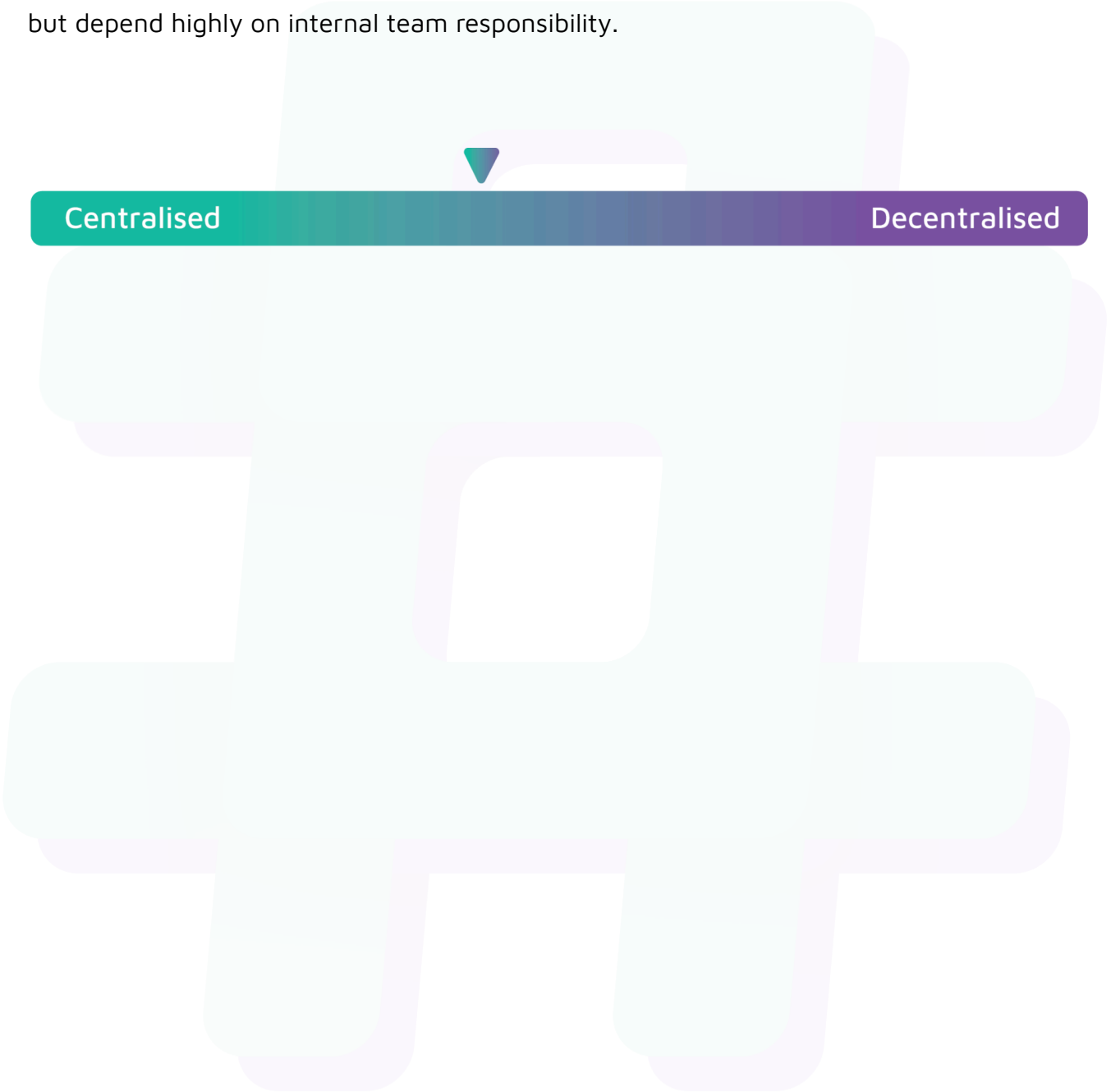
Status

Resolved

Centralisation

The UNIT Technologies project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the UNIT Technologies project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved or acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.